DIAGRAMA DE CLASES PRELIMINAR

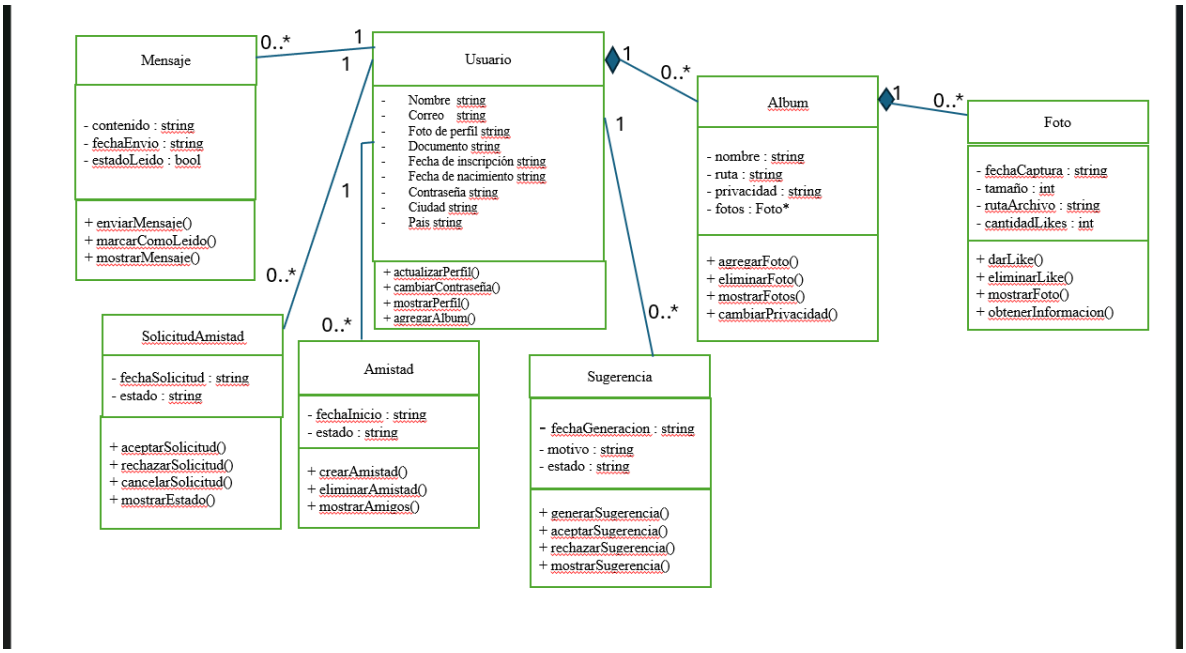
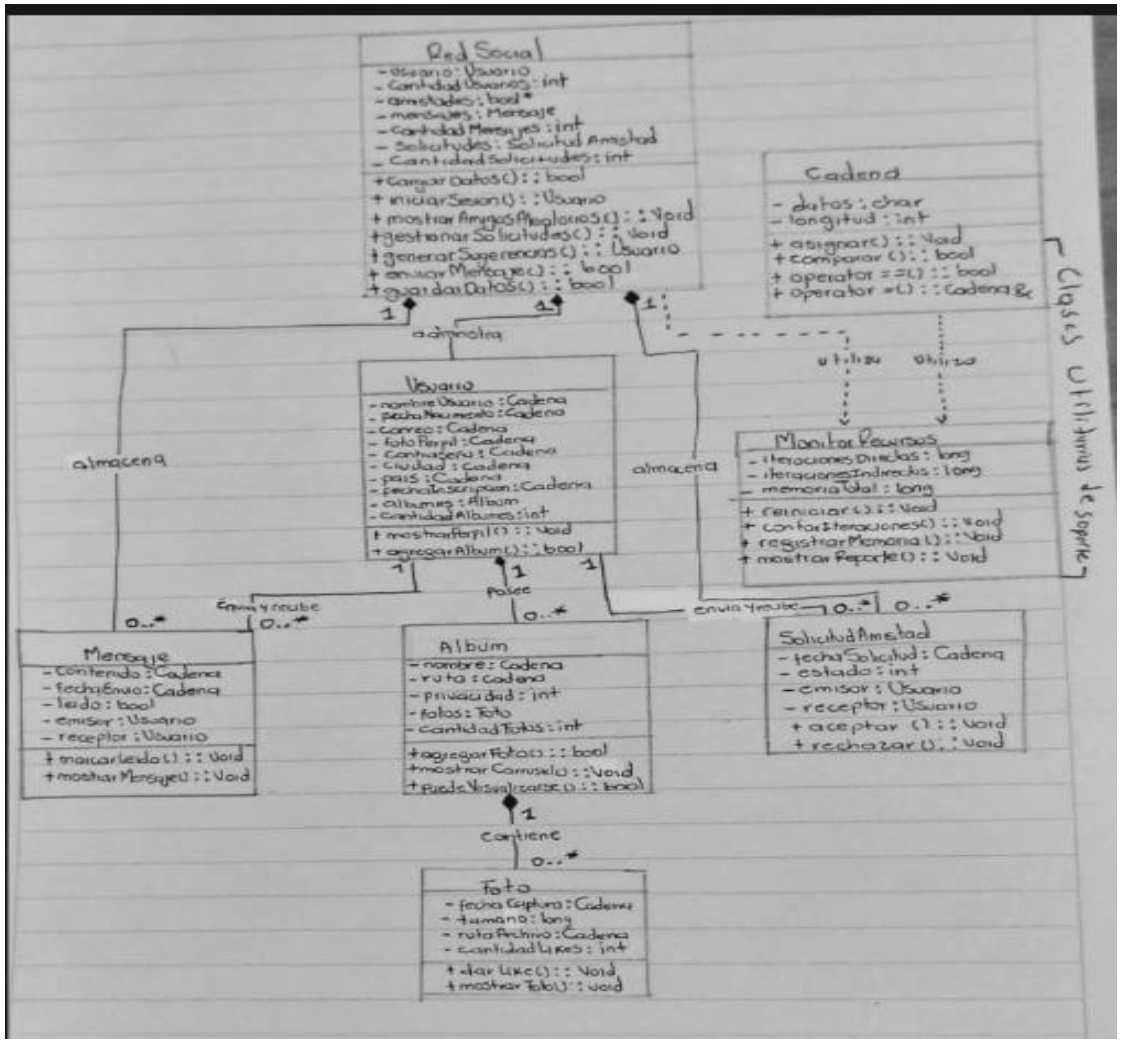


DIAGRAMA DE CLASES DEFINITIVO



Camios realizados y sustentados: Se realizan variaciones principalmente por el uso preliminar de una restricción como era (string) ya que pertenece a la STL y en en desafio no se permite este tipo de librerías el segundo cambio importante fue por requerimientos y eficiencia al quitar el (string) se genera la necesidad de crear una librería propia "Tanto Cadena como MonitorRecursos son clases, pero de naturaleza utilitaria y de soporte. Cadena es nuestra propia abstracción orientada a objetos para manejar texto sin recurrir a la STL, mientras que MonitorRecursos es el módulo

transversal encargado de auditar la huella de memoria y la complejidad computacional en iteraciones exigida en la rúbrica."

Justificación de la Eliminación de las Clases Amistad y Sugerencia, y Conservación de SolicitudAmistad

1. Eliminación de la clase Amistad en favor de una Matriz de Adyacencia

- **Problema del diseño preliminar:**

En la concepción inicial, se modeló una clase Amistad para representar la conexión entre dos usuarios. Sin embargo, en un entorno de red social con más de 1000 usuarios, modelar cada relación como un objeto individual en memoria dinámica genera una fragmentación severa del Heap, sobrecosto por cabeceras de asignación (*overhead* de punteros) y obliga a búsquedas secuenciales de complejidad $O(M)$ (donde M es el número de amistades) para comprobar si dos personas son amigas.

- **Decisión técnica adoptada:**

La relación de amistad es simétrica y no dirigida. Por ende, la entidad Amistad se eliminó conceptualmente y se trasladó al controlador (RedLeon) bajo la estructura de una **matriz de adyacencia dinámica bidimensional (bool** amistades)**.

- **Beneficios de ingeniería de software:**
    - **Acceso en tiempo constante $O(1)$:** Verificar si el usuario i y el usuario j son amigos toma una sola instrucción en memoria (amistades[i][j]).
    - **Cálculo masivo de amigos en común:** El conteo de intersecciones entre amigos se reduce a una operación lineal $O(N)$ sobre un vector contiguo de memoria, optimizando el rendimiento de la CPU gracias a la localidad espacial en la memoria caché.
    - **Ahorro de memoria:** Un booleano por nodo ocupa sensiblemente menos espacio que una instancia de objeto con metadatos.

2. Conservación de la clase SolicitudAmistad

- **Por qué SolicitudAmistad sí debe existir como clase:**

A diferencia de la amistad consolidada (que es un estado de relación estático), una solicitud es un **proceso transaccional y asimétrico**:

  - Posee dirección de negocio: un usuario **emisor** y un usuario **receptor**.
  - Posee una estampa temporal (fechaSolicitud).
  - Posee un ciclo de vida con transición de estados (0: Pendiente, 1: Aceptada, 2: Rechazada).

- **Coordinación con el sistema:**

SolicitudAmistad se mantiene como entidad de dominio responsable de gestionar las invitaciones activas. Cuando el receptor ejecuta el método aceptar(), la solicitud concluye su ciclo y notifica al controlador para que active la conexión bidireccional en la matriz de adyacencia (amistades[emisor][receptor] = true).

3. Eliminación de la clase Sugerencia

- **Violación de principios de diseño:**

En el modelo inicial se contempló una clase Sugerencia. No obstante, en la Programación Orientada a Objetos (POO), una sugerencia de amistad no constituye una entidad persistente ni posee identidad de dominio propia; es únicamente el **resultado efímero de una consulta computacional**.

- **Decisión técnica adoptada:**

Crear objetos Sugerencia implicaba reservar y liberar memoria de forma innecesaria en cada consulta. Por ello, se eliminó la clase y la responsabilidad se asignó al controlador mediante el método algorítmico generarSugerencias(Usuario* u, int k).

- **Beneficios:**

El cálculo analiza directamente las filas de la matriz de adyacencia de los usuarios no conectados, clasifica en tiempo de ejecución los candidatos con mayor índice de coincidencias (amigos en común) y presenta los resultados al usuario sin sobrecargar el recolector o la gestión manual de memoria dinámica.